

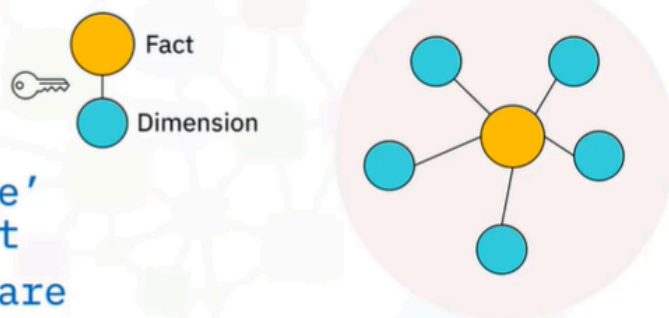
Data Modeling using Star and Snowflake Schema

Star schemas

- Keys connect facts with dimensions

Star schema

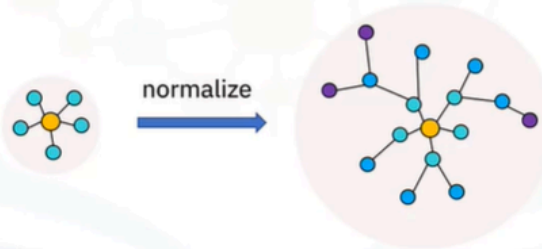
- Dimensions 'radiate' from a central fact
- Graph whose edges are relations between facts and dimensions



- Recall that a fact table contains foreign keys that refer to the primary keys of dimension tables. The idea of a star schema is based on the way a set of dimension tables can be visualized, or modeled, as radiating from a central fact table, linked by these keys. A star schema is thus a graph, whose nodes are fact and dimension tables, and whose edges are the relations between those tables. Star schemas are commonly used to develop specialized data warehouses called "data marts."

Snowflake schemas

- Snowflake schemas are normalized star schemas
- Dimension tables split into child tables
- Snowflakes don't need to be fully normalized

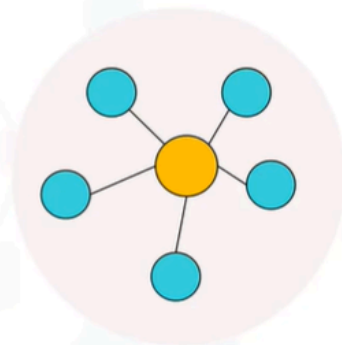


- Snowflake schemas are a generalization of star schemas and can be seen as normalized star schemas. Normalization means separating the levels or hierarchies of a dimension table into separate child tables. A schema need not be fully normalized to be considered a snowflake, so long as at least one of its dimensions has its levels separated.

Modeling with a star schema

Design considerations

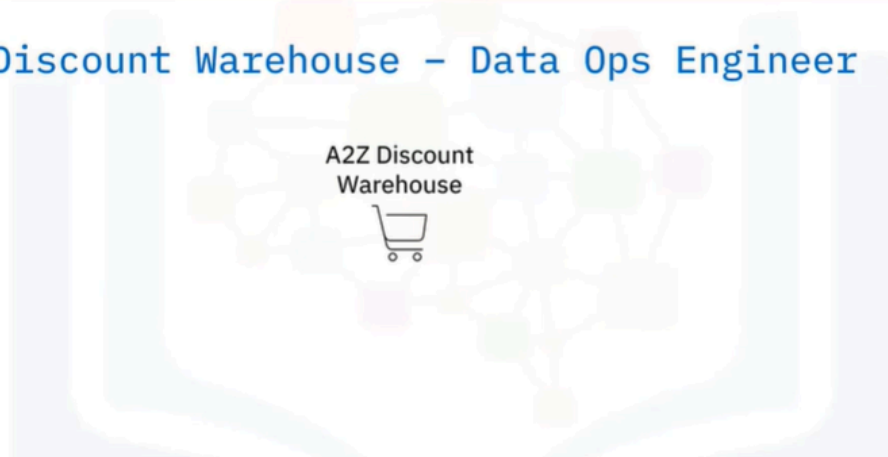
1. Select a business process
2. Choose level of detail
3. Identify the dimensions
4. Identify the facts



- Let's look at some general principles you need to consider when designing a data model for a star schema.
 - The first step involves selecting a business process as the basis for what you want to model. You might be interested in processes such as sales, manufacturing, or supply chain logistics.
 - In step two, you need to choose a granularity, which is the level of detail that you need to capture. Are you interested in coarse-grained information such as annual regional sales numbers? Or, maybe you want to drill down into monthly sales performance by salesperson.
 - Next in the process, you need to identify the dimensions. These may include attributes such as the date and time, and names of people, places, and things.
 - The final consideration in designing star schemas is to identify the facts. These are the things being measured in the business process.

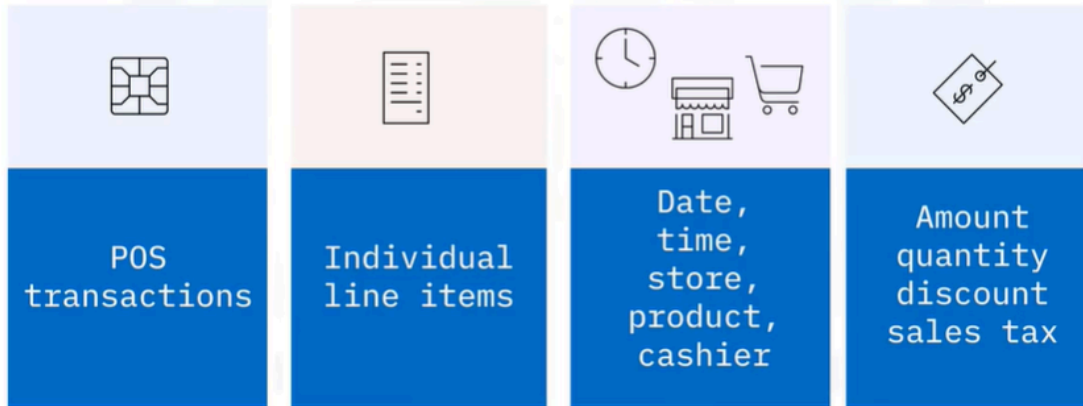
Data Warehouse Architecture

A2Z Discount Warehouse – Data Ops Engineer



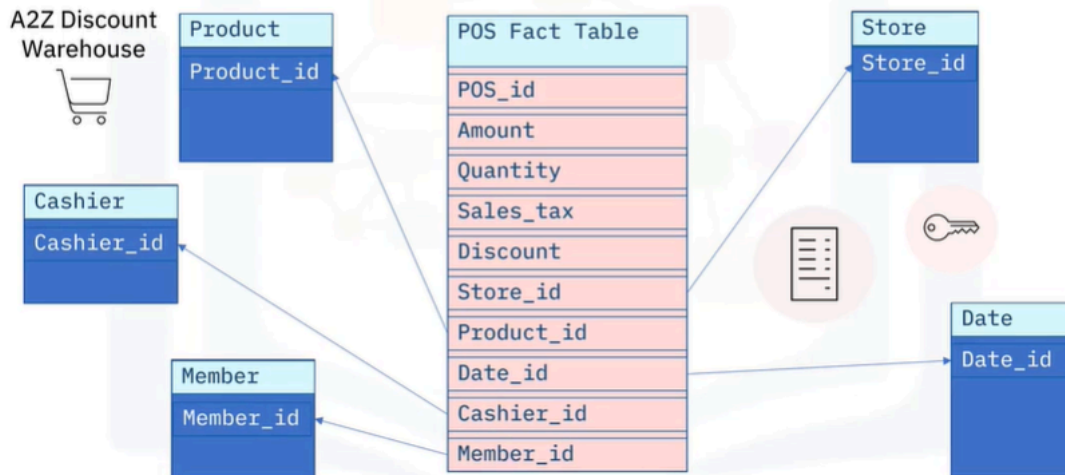
Data Warehouse Architecture

A2Z Discount Warehouse – Data Ops Engineer



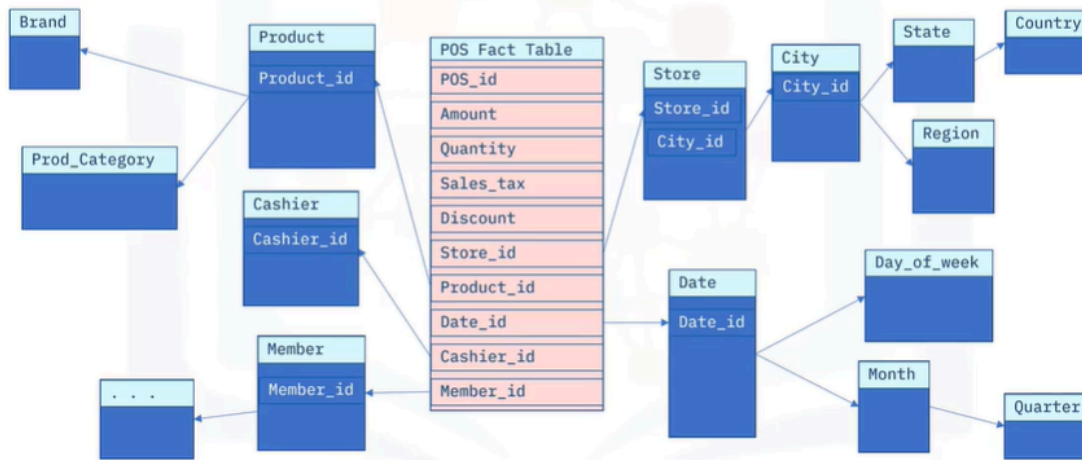
- Let's apply these considerations to a scenario. Imagine, for example, that you are a data engineer helping to lay out the data ops for a new store called "A to Z Discount Warehouse."
- They would like you to develop a data plan to capture everyday POS, or point-of-sales transactions that happen at the till, where customers have their items scanned and pay for them. Thus, "point-of-sale transactions" is the business process that you want to model.
- The finest granularity you can expect to capture from POS transactions comes from the individual line items, which is included in the detailed information you can see on a typical store receipt. This is precisely what "A to Z" is interested in capturing.
- The next step in the process is to identify the dimensions. These include attributes such as the date and time of the purchase, the store name, the products purchased, and the cashier who processed the items. You might add other dimensions, like "payment method," whether the line item is a return or a purchase, and perhaps a "customer membership number."
- Now it's time to consider the facts. Thus, you identify facts such as the amount for each item's price, the quantity of each product sold, any discounts applied to the sale, and the sales tax applied. Other facts to consider include environmental fees, or deposit fees for returnable containers.

Point-of-sale star



- Now you are ready to start building your star schema for "A to Z Discount Warehouse." At the center of your star schema sits a "point-of-sales fact table," which contains a unique ID, called "P O S ID," for each line item in the transaction, plus the following facts, or measures: the amount of the transaction in dollars, the quantity, or number of items involved, the sales tax, and any discount applied. There may be other facts to include, but these can be added later as you discover them.
- Each line item from a sales transaction has many dimensions associated with it. You include them as foreign keys in your fact table, or as links to the primary keys of your dimension tables. For example, the name of the store at which the item was sold is kept in a dimension table called "store," which is identified in the fact-table by the value of the foreign "Store ID" key, which is the primary key for the Store table. Product information is stored in the Product table, which is uniquely identified by the "ProductID" key. Similarly, the date of the transaction is keyed by the "Date ID," which cashier entered the transaction is keyed by the "Cashier ID," and which member was involved is indicated by the "Member ID." This illustrates what a star schema might look like.

Point-of-sale snowflake



- Let's see how you can use normalization to extend your star schema to a snowflake schema. Starting with your star schema, you can extract some of the details of the dimension tables into their own separate dimension tables, creating a hierarchy of tables.
- A separate city table can be used to record which city the store is in, while a foreign 'city id' key would be included in the 'Store' table to maintain the link. You might also have tables and keys for the city's state or province, and a pre-defined sales region for the store, and for which country the store resides in. We've left out the associated keys for simplicity. We can continue to normalize other dimensions, like the product's brand, and a "product category" that it belongs to, the day of week and the month corresponding to the date, plus the quarter, and so on. This normalized version of the star schema is called a snowflake schema, due to its multiple layers of branching which resembles a snowflake pattern. Much like how pointers are used to point to memory locations in computing, normalization reduces the memory footprint of the data.

Summary

In this video, you learned that:

- Facts and dimension tables, linked by keys, form star or snowflake modeling schemas
- Design considerations include identifying a business process, its granularity, and its facts and dimensions
- Snowflake schemas are normalized star schemas
- Normalization involves separating dimensions into their hierarchies

what trade-off between Star schema and Snowflake schema?

The trade-offs between **star schema** and **snowflake schema** primarily revolve around **complexity** and **performance**. Here's a brief comparison:

Star Schema

- **Simplicity:** The star schema is straightforward and easy to understand. All dimension tables are directly linked to the fact table.
- **Performance:** Queries tend to be faster because fewer joins are needed. This is beneficial for reporting and analysis.
- **Redundancy:** It may have some data redundancy since dimension tables can contain repeated information.

Snowflake Schema

- **Complexity:** The snowflake schema is more complex as it normalizes dimension tables into multiple related tables. This can make it harder to understand.
- **Performance:** Queries may be slower due to the need for more joins between tables. However, it can save storage space by reducing redundancy.

- **Data Integrity:** It maintains better data integrity and reduces redundancy, as related data is stored in separate tables.

Summary

- **Use Star Schema** when you need fast query performance and simplicity.
- **Use Snowflake Schema** when you want to save storage space and maintain data integrity, even if it means more complex queries.

What factors influence the choice between star and snowflake schemas?

Several factors influence the choice between **star** and **snowflake schemas** in data modeling. Here are the key considerations:

1. Query Performance

- **Star Schema:** Generally offers faster query performance due to fewer joins.
- **Snowflake Schema:** May result in slower queries because of the increased number of joins.

2. Data Redundancy

- **Star Schema:** Can lead to data redundancy since dimension tables may contain repeated information.
- **Snowflake Schema:** Reduces redundancy by normalizing dimension tables into multiple related tables.

3. Complexity

- **Star Schema:** Simpler and easier to understand, making it user-friendly for reporting.
- **Snowflake Schema:** More complex due to multiple related tables, which can be harder to navigate.

4. Storage Efficiency

- **Star Schema:** May require more storage space due to redundancy.
- **Snowflake Schema:** More storage-efficient as it minimizes data duplication.

5. Data Integrity

- **Star Schema:** May have challenges with data integrity due to redundancy.
- **Snowflake Schema:** Better maintains data integrity by organizing related data into separate tables.

6. Business Requirements

- **Star Schema:** Suitable for environments where quick reporting and analysis are prioritized.
- **Snowflake Schema:** Better for complex analytical needs where detailed data relationships are important.

7. User Skill Level

- **Star Schema:** More accessible for users who may not have advanced SQL skills.
- **Snowflake Schema:** May require more advanced SQL knowledge to navigate the complex structure.

Considering these factors can help determine which schema best fits the specific needs of a data warehousing project.

in business scenario what factors you should consider to choose between star schema or snowflake schema?

In a business scenario, choosing between a **star schema** and a **snowflake schema** involves considering several specific factors:

1. Reporting Needs

- **Star Schema:** If quick and straightforward reporting is essential, a star schema is preferable due to its simplicity and faster query performance.

- **Snowflake Schema:** If detailed analysis requiring complex queries is needed, a snowflake schema may be more suitable.

2. Data Volume and Complexity

- **Star Schema:** Works well with moderate data volumes and simpler relationships.
- **Snowflake Schema:** Better for large datasets with complex relationships, as it organizes data more efficiently.

3. User Expertise

- **Star Schema:** Ideal for users with limited technical skills, as it is easier to understand and navigate.
- **Snowflake Schema:** More appropriate for users with advanced SQL skills who can handle complex queries.

4. Data Maintenance and Updates

- **Star Schema:** May require more effort to maintain due to potential data redundancy.
- **Snowflake Schema:** Easier to maintain data integrity and consistency due to normalization.

5. Storage Considerations

- **Star Schema:** May consume more storage space due to redundancy.
- **Snowflake Schema:** More storage-efficient, making it suitable for organizations with limited storage resources.

6. Performance Requirements

- **Star Schema:** Preferred when performance is critical, as it typically offers faster query response times.
- **Snowflake Schema:** May be acceptable if the business can tolerate slower query performance for the sake of data integrity.

7. Business Processes

- **Star Schema:** Suitable for straightforward business processes where the focus is on summarizing data.

- **Snowflake Schema:** Better for complex business processes that require detailed analysis of multiple dimensions.

8. Future Scalability

- **Star Schema:** May become cumbersome as data grows and relationships become more complex.
- **Snowflake Schema:** More adaptable to changes and growth in data complexity over time.

By evaluating these factors in the context of specific business needs, organizations can make a more informed decision on which schema to implement.

What is the main difference between star and snowflake schemas?

The main difference between **star schema** and **snowflake schema** lies in their structure and organization of data:

Star Schema

- **Structure:** Consists of a central **fact table** connected directly to multiple **dimension tables**. The dimension tables are typically denormalized, meaning they contain all relevant attributes in a single table.
- **Simplicity:** Easier to understand and navigate due to its straightforward design.
- **Performance:** Generally offers faster query performance because fewer joins are required when querying data.
- **Redundancy:** May have data redundancy since dimension tables can contain repeated information.

Snowflake Schema

- **Structure:** Similar to a star schema but with normalized dimension tables. This means that dimension tables are split into multiple related tables, creating a more complex structure.

- **Complexity:** More complex and can be harder to understand due to the multiple layers of tables.
- **Performance:** May result in slower query performance because more joins are needed to retrieve data.
- **Data Integrity:** Reduces redundancy and maintains better data integrity by organizing related data into separate tables.

Summary

- **Star Schema:** Simple, faster queries, potential redundancy.
- **Snowflake Schema:** Complex, slower queries, reduced redundancy.

What are the advantages of using a snowflake schema over a star schema?

Using a **snowflake schema** offers several advantages over a **star schema**:

1. Reduced Data Redundancy

- Snowflake schemas normalize dimension tables, which minimizes data duplication. This leads to a more efficient use of storage.

2. Improved Data Integrity

- By organizing related data into separate tables, snowflake schemas help maintain data integrity. Changes to data in one table automatically reflect in related tables, reducing the risk of inconsistencies.

3. Better Organization of Data

- The hierarchical structure of snowflake schemas allows for a more organized representation of data. This can be beneficial for complex datasets with multiple levels of detail.

4. Flexibility for Changes

- Snowflake schemas can adapt more easily to changes in business requirements. New dimensions or attributes can be added without significantly

altering the existing structure.

5. Efficient Querying for Complex Analyses

- For analytical queries that require detailed relationships between dimensions, snowflake schemas can provide a clearer structure, making it easier to perform complex analyses.

6. Optimized Storage Usage

- By reducing redundancy, snowflake schemas can lead to lower storage costs, which is particularly important for large datasets.

7. Support for Hierarchical Data

- Snowflake schemas are well-suited for representing hierarchical relationships within dimensions, such as geographical data (country, state, city).

These advantages make snowflake schemas a suitable choice for organizations that prioritize data integrity, efficient storage, and the ability to handle complex analytical needs.

When worked with data what key should consider to choose between star schema or snowflake schema ?

When working with data, several key factors should be considered to choose between a **star schema** and a **snowflake schema**:

1. Query Performance

- **Star Schema:** If fast query performance is a priority, especially for reporting, a star schema is often preferred due to fewer joins.
- **Snowflake Schema:** If the data model requires complex queries that benefit from normalized data, a snowflake schema may be more appropriate.

2. Data Redundancy and Integrity

- **Star Schema:** Consider if data redundancy is acceptable. Star schemas may have duplicated data in dimension tables.

- **Snowflake Schema:** If maintaining data integrity and minimizing redundancy is crucial, a snowflake schema is better.

3. Complexity of Data Relationships

- **Star Schema:** Suitable for simpler data relationships where ease of understanding is important.
- **Snowflake Schema:** Better for complex relationships that require a more detailed and organized structure.

4. User Skill Level

- **Star Schema:** Ideal for users with limited technical skills, as it is easier to navigate and understand.
- **Snowflake Schema:** More suitable for users with advanced SQL skills who can handle complex queries.

5. Storage Considerations

- **Star Schema:** May require more storage space due to redundancy.
- **Snowflake Schema:** More storage-efficient, making it suitable for large datasets.

6. Business Requirements

- **Star Schema:** If the focus is on summarizing data for quick insights, a star schema is appropriate.
- **Snowflake Schema:** If detailed analysis and reporting on multiple dimensions are needed, a snowflake schema is preferable.

7. Future Scalability

- **Star Schema:** May become cumbersome as data grows and relationships become more complex.
- **Snowflake Schema:** More adaptable to changes and growth in data complexity over time.

By evaluating these key factors in the context of your specific data needs, you can make a more informed decision on which schema to implement.